Milestone 4

Christopher Zion

Grand Canyon University

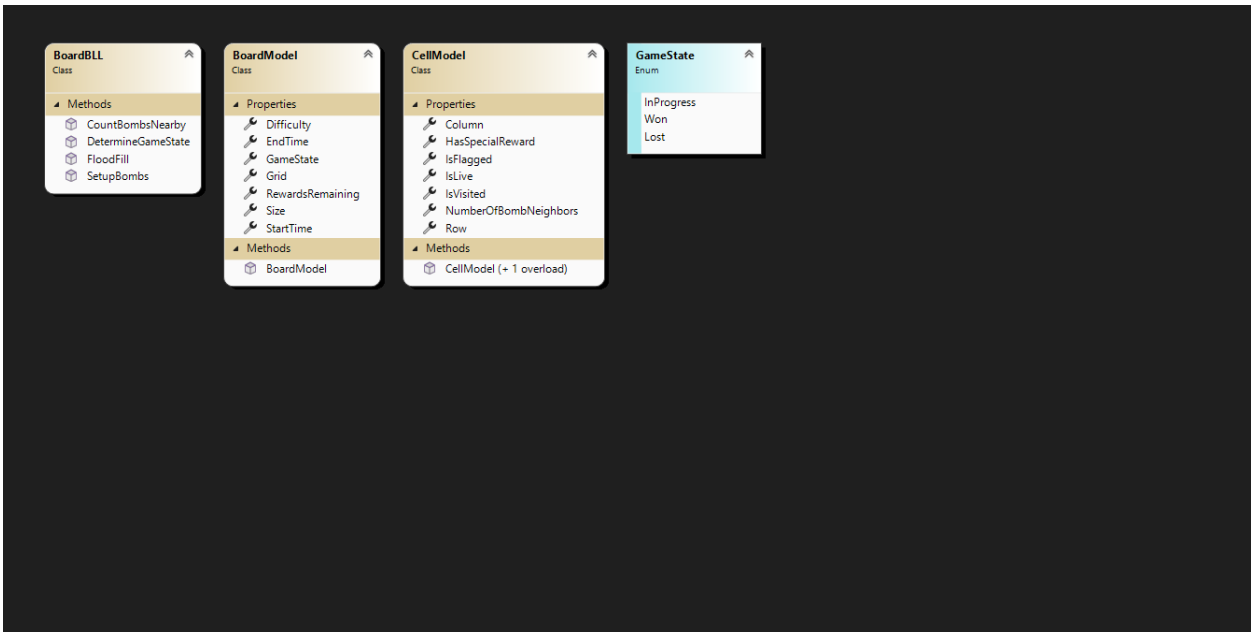
Course Number: CST-250

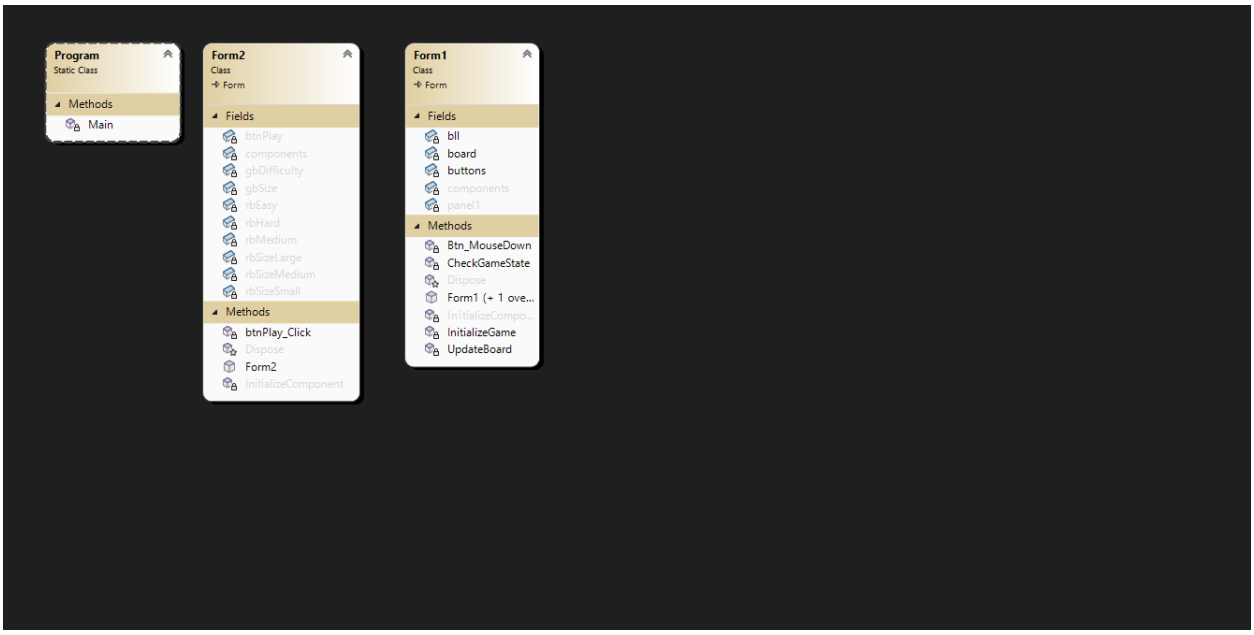
Github Repo:

https://github.com/cziongcu/Minesweeper-Milestone

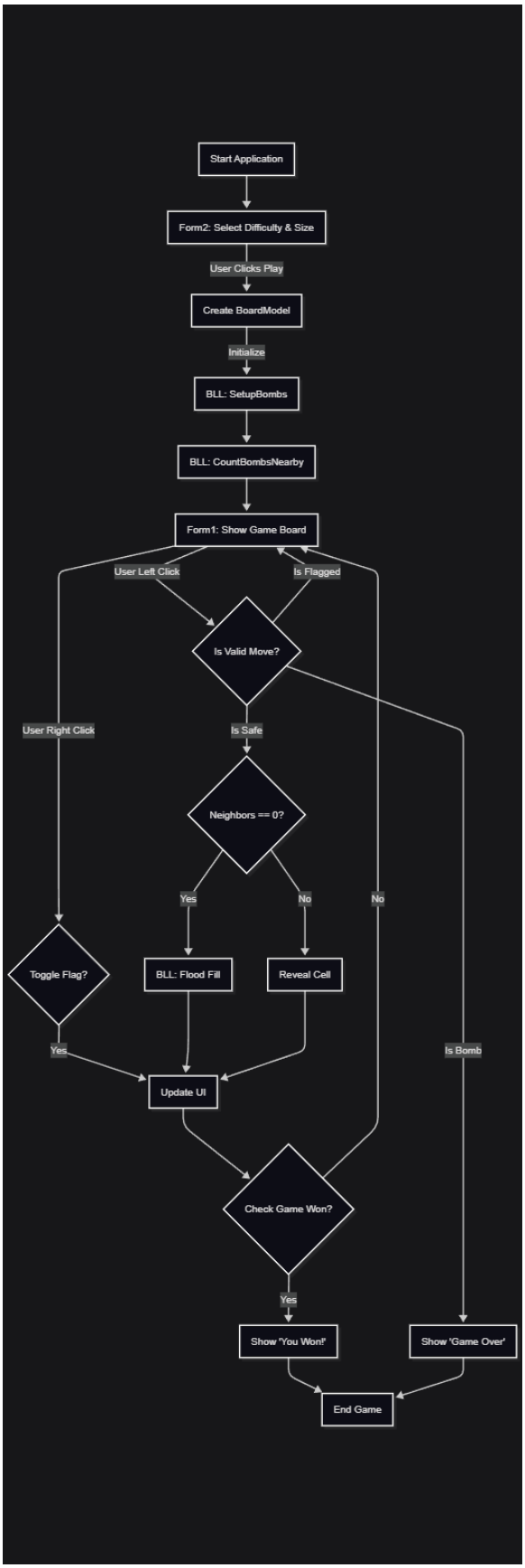
UML Diagrams:

Program
Class
Methods
Main
PrintAnswers
PrintBoard

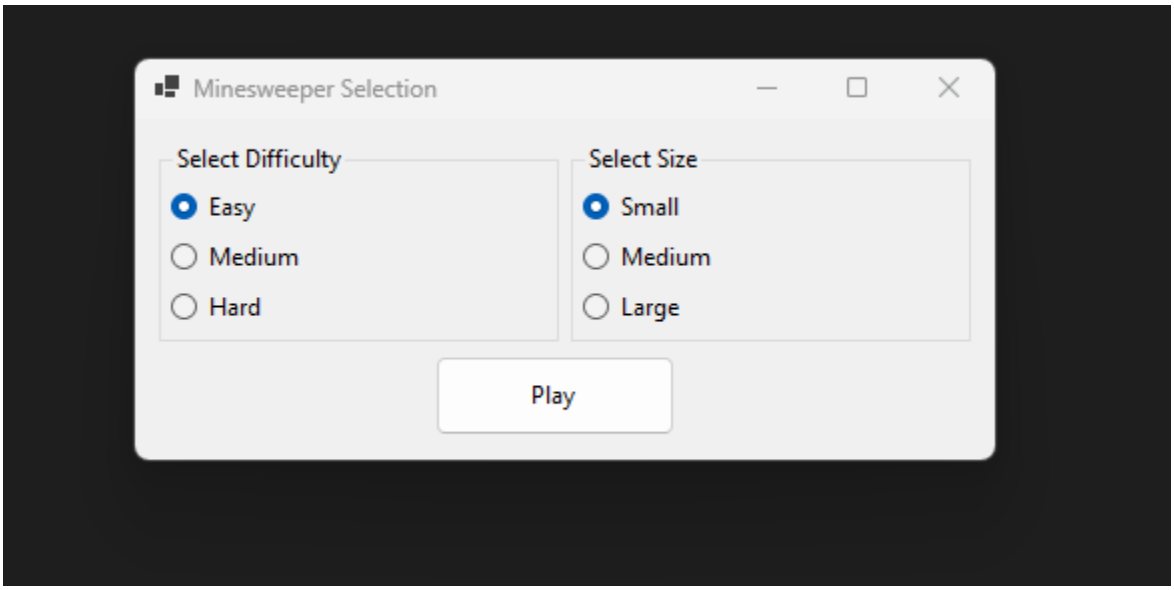

BoardBLL
Class
Methods
CountBombsNearby
DetermineGameState
FloodFill
SetupBombs
BoardModel
Class
Properties
Difficulty
EndTime
GameState
Grid
RewardsRemaining
Size
StartTime
Methods
BoardModel
CellModel
Class
Properties
Column
HasSpecialReward
IsFlagged
IsLive
IsVisited
NumberOfBombNeighbors
Row
Methods
CellModel (+ 1 overload)
GameState
Enum
InProgress
Won
Lost


Program
Static Class
Methods
Main
Form2
Class
Form
Fields
btnPlay
components
gbDifficulty
gbSize
rbEasy
rbHard
rbMedium
rbSizeLarge
rbSizeMedium
rbSizeSmall
Methods
btnPlay_Click
Dispose
Form2
InitializeComponent
Form1
Class
Form
Fields
bll
board
buttons
components
panel1
Methods
Btn_MouseDown
CheckGameState
Dispose
Form1 (+ 1 ove...
InitializeCompo...
InitializeGame
UpdateBoard

Flowcharts:


Start Application
Form2: Select Difficulty & Size
User Clicks Play
Create BoardModel
Initialize
BLL: SetupBombs
BLL: CountBombsNearby
Form1: Show Game Board
User Left Click
Is Flagged
Is Valid Move?
User Right Click
Is Safe
Neighbors == 0?
Yes
No
No
Toggle Flag?
BLL: Flood Fill
Reveal Cell
Yes
Is Bomb
Update UI
Check Game Won?
Yes
Show 'You Won!'
Show 'Game Over'
End Game

Pictures:



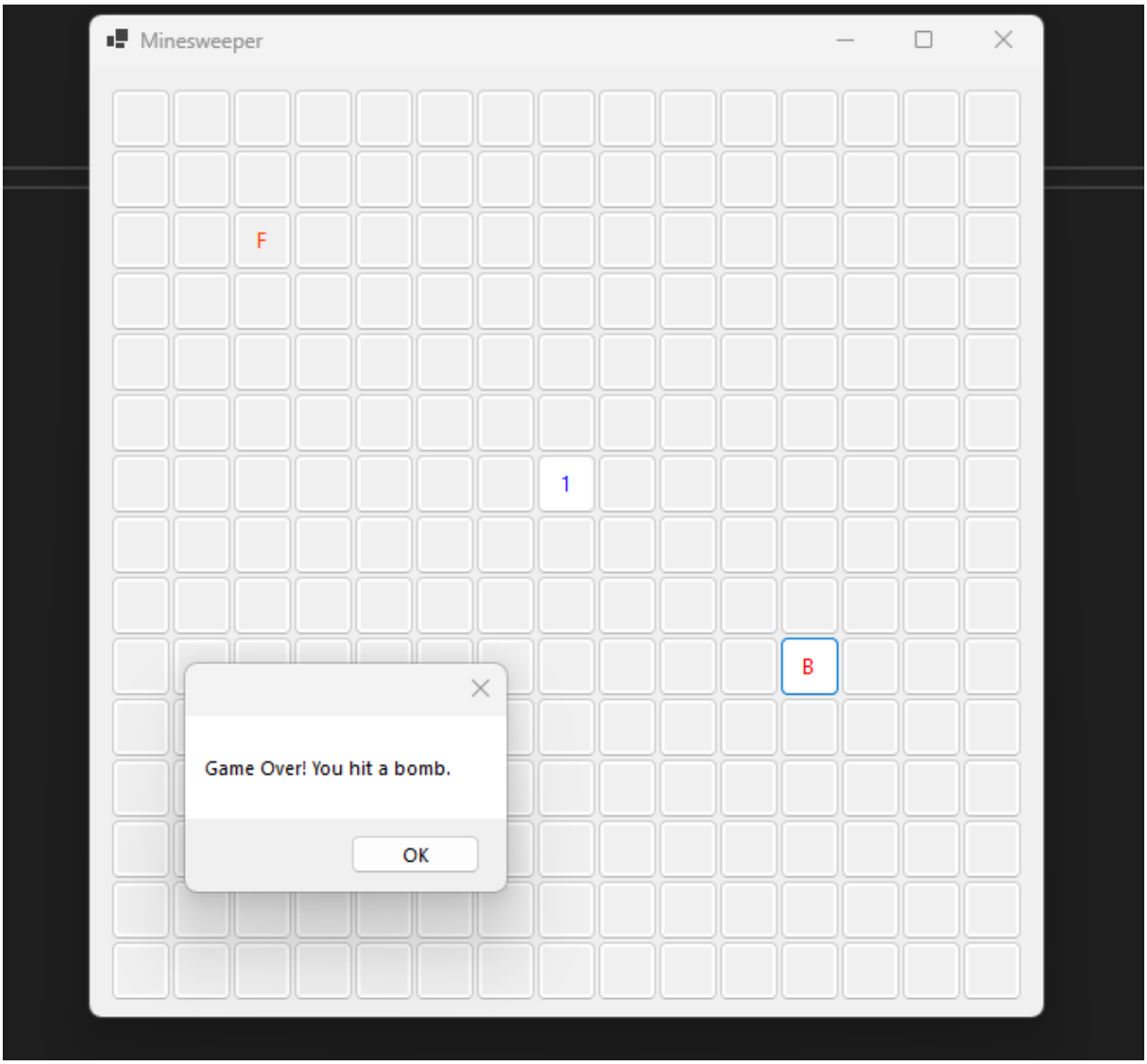
```
using System.Windows.Forms;
using MineSweeperClassLib.Models;

namespace MinesweeperGUI
{
    3 references
    public partial class Form2 : Form
    {
        1 reference
        public Form2()
        {
            InitializeComponent();
        }

        1 reference
        private void btnPlay_Click(object sender, EventArgs e)
        {
            // Default values
            int difficulty = 10; // Easy
            int size = 10; // Small

            // Check difficulty
            if (rbMedium.Checked) difficulty = 20;
            else if (rbHard.Checked) difficulty = 30;

            // Check size
            if (rbSizeMedium.Checked) size = 15;
            else if (rbSizeLarge.Checked) size = 20;

            // Create board
            BoardModel board = new BoardModel(size);
            board.Difficulty = difficulty;

            // Hide this form and show game board
            Form1 gameForm = new Form1(board);
            gameForm.FormClosed += (s, args) => this.Close();
            this.Hide();
            gameForm.Show();
        }
    }
}
```

```
private void UpdateBoard()
{
    for (int r = 0; r < board.Size; r++)
    {
        for (int c = 0; c < board.Size; c++)
        {
            Button btn = buttons[r, c];
            CellModel cell = board.Grid[r, c];

            if (cell.IsVisited)
            {
                btn.BackColor = Color.White;
                if (cell.IsLive)
                {
                    btn.Text = "B";
                    btn.ForeColor = Color.Red;
                }
                else
                {
                    if (cell.NumberOfBombNeighbors > 0)
                    {
                        btn.Text = cell.NumberOfBombNeighbors.ToString();

                        // Colors like the guide
                        switch(cell.NumberOfBombNeighbors)
                        {
                            case 1: btn.ForeColor = Color.Blue; break;
                            case 2: btn.ForeColor = Color.Green; break;
                            case 3: btn.ForeColor = Color.Red; break;
                            default: btn.ForeColor = Color.DarkRed; break;
                        }
                    }
                    else
                    {
                        btn.Text = "";
                    }
                }
            }
            else
            {
                // Unvisited
                btn.BackColor = SystemColors.ButtonFace;
                if (cell.IsFlagged)
                {
                    btn.Text = "F";
                    btn.ForeColor = Color.OrangeRed;
                }
                else
```

```
    }

    1 reference
    private void Btn_MouseDown(object sender, MouseEventArgs e)
    {
        Button btn = sender as Button;
        Point p = (Point)btn.Tag;
        int r = p.X;
        int c = p.Y;

        if (e.Button == MouseButtons.Right)
        {
            // Right click to flag
            board.Grid[r, c].IsFlagged = !board.Grid[r, c].IsFlagged;
        }
        else if (e.Button == MouseButtons.Left)
        {
            // Left click to visit
            if (board.Grid[r, c].IsFlagged) return;

            board.Grid[r, c].IsVisited = true;

            // Hit a bomb?
            if (board.Grid[r, c].IsLive)
            {
                UpdateBoard();
                MessageBox.Show("Game Over! You hit a bomb.");
                this.Close();
                return;
            }

            // Flood fill
            if (board.Grid[r, c].NumberOfBombNeighbors == 0)
            {
                bll.FloodFill(r, c, board);
            }
        }

        UpdateBoard();
        CheckGameState();
    }

    3 references
    private void UpdateBoard()
    {
        for (int r = 0; r < board.Size; r++)
        {
            for (int c = 0; c < board.Size; c++)
```

```
    2 references
    private void InitializeGame()
    {
        // Setup backend
        bll.SetupBombs(board);
        bll.CountBombsNearby(board);

        // Setup UI
        int buttonSize = 35;
        this.Width = board.Size * buttonSize + 40;
        this.Height = board.Size * buttonSize + 60;

        buttons = new Button[board.Size, board.Size];
        panel1.Width = board.Size * buttonSize;
        panel1.Height = board.Size * buttonSize;

        for (int r = 0; r < board.Size; r++)
        {
            for (int c = 0; c < board.Size; c++)
            {
                Button btn = new Button();
                btn.Width = buttonSize;
                btn.Height = buttonSize;
                btn.Location = new Point(c * buttonSize, r * buttonSize);
                btn.Tag = new Point(r, c);
                btn.MouseDown += Btn_MouseDown;
                panel1.Controls.Add(btn);
                buttons[r, c] = btn;
            }
        }
    }

    1 reference
    private void Btn_MouseDown(object sender, MouseEventArgs e)
    {
        Button btn = sender as Button;
        Point p = (Point)btn.Tag;
        int r = p.X;
        int c = p.Y;

        if (e.Button == MouseButtons.Right)
        {
            // Right click to flag
            board.Grid[r, c].IsFlagged = !board.Grid[r, c].IsFlagged;
        }
        else if (e.Button == MouseButtons.Left)
        {
            // Left click to visit
```

```
/*
 * Christopher Zion
 * CST-250
 * Milestone 4
 */
using System;
using System.Drawing;
using System.Windows.Forms;
using MineSweeperClassLib.BLL;
using MineSweeperClassLib.Models;

namespace MinesweeperGUI
{
    5 references
    public partial class Form1 : Form
    {
        private BoardModel board;
        private BoardBLL bll = new BoardBLL();
        private Button[,] buttons;

        1 reference
        public Form1(BoardModel boardModel)
        {
            InitializeComponent();
            this.board = boardModel;
            InitializeGame();
        }

        0 references
        public Form1()
        {
            InitializeComponent();
             // Default for designer or empty init
             board = new BoardModel(10);
             InitializeGame();
        }

        2 references
        private void InitializeGame()
        {
            // Setup backend
            bll.SetupBombs(board);
            bll.CountBombsNearby(board);

            // Setup UI
            int buttonSize = 35;
            this.Width = board.Size * buttonSize + 40;
            this.Height = board.Size * buttonSize + 60;
```



Summary

In this milestone, we moved from the console to a real GUI using Windows Forms, which makes the game actually playable with a mouse. The biggest thing we learned was Event-Driven Programming, which just means the code sits there and waits for an event—like me clicking a button instead of running straight through from start to finish. We also got practice with controls like Buttons and Panels, dynamically creating a grid of them in the code to match our board size. It really showed why we kept our logic separate in the BLL earlier, because it was super easy to just plug that existing code into the new forms to make the buttons change colors and show numbers.